

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 11

TITLE:

Design and develop a Cloud-Based Industrial DG Monitoring and Analytics System. The system collects and analyzes diesel generator performance data using cloud technologies for remote monitoring applications.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

Diesel Generators (DG) are widely used as backup power sources in industries, hospitals, commercial buildings, and other critical applications. Continuous monitoring of DG parameters helps improve generator performance, reduce unnecessary fuel consumption, and identify abnormal operating conditions. The proposed **Cloud-Based Industrial DG Monitoring and Analytics System** uses an **ESP32 microcontroller, ACS712 current sensor, voltage sensor module, DHT11 sensor, OLED display, and cloud platform** to monitor DG performance parameters remotely. The system collects voltage, current, temperature, and other operating information. The collected data is displayed locally on the OLED and transmitted to the cloud for remote monitoring and analysis.

PROBLEM STATEMENT

- Manual monitoring of DG parameters.
- Difficulty in continuous DG monitoring.
- Lack of remote monitoring facilities.
- Difficulty in identifying abnormal voltage and current conditions.
- Increased risk of DG overheating.
- Difficulty in maintaining performance records.
- No real-time cloud-based data visualization.
- Delayed detection of abnormal operating conditions.

PROPOSED SOLUTION

The sensors continuously measure DG operating parameters. The ESP32 processes the sensor data and displays the values on an OLED display. The data is also transmitted to a cloud platform for remote monitoring and analytics.

The system can automatically monitor:

- DG voltage
- DG current
- DG temperature
- Power-related parameters
- Operating condition
- Abnormal conditions
- Real-time cloud data

WORKING PRINCIPLE

- The voltage sensor measures the DG output voltage.
- The ACS712 current sensor measures the load current.
- The DHT11 sensor measures temperature and humidity.
- The ESP32 receives the sensor signals.
- The ESP32 processes the collected data.
- The measured values are displayed on the OLED.
- The ESP32 connects to Wi-Fi.
- The sensor data is uploaded to the cloud.
- The cloud dashboard displays the DG parameters remotely.
- The collected data can be analyzed to identify abnormal operating conditions.

SYSTEM ARCHITECTURE

DG → Voltage Sensor → ESP32 → DG Load → ACS712 → ESP32 → Environment → DHT11 → ESP32

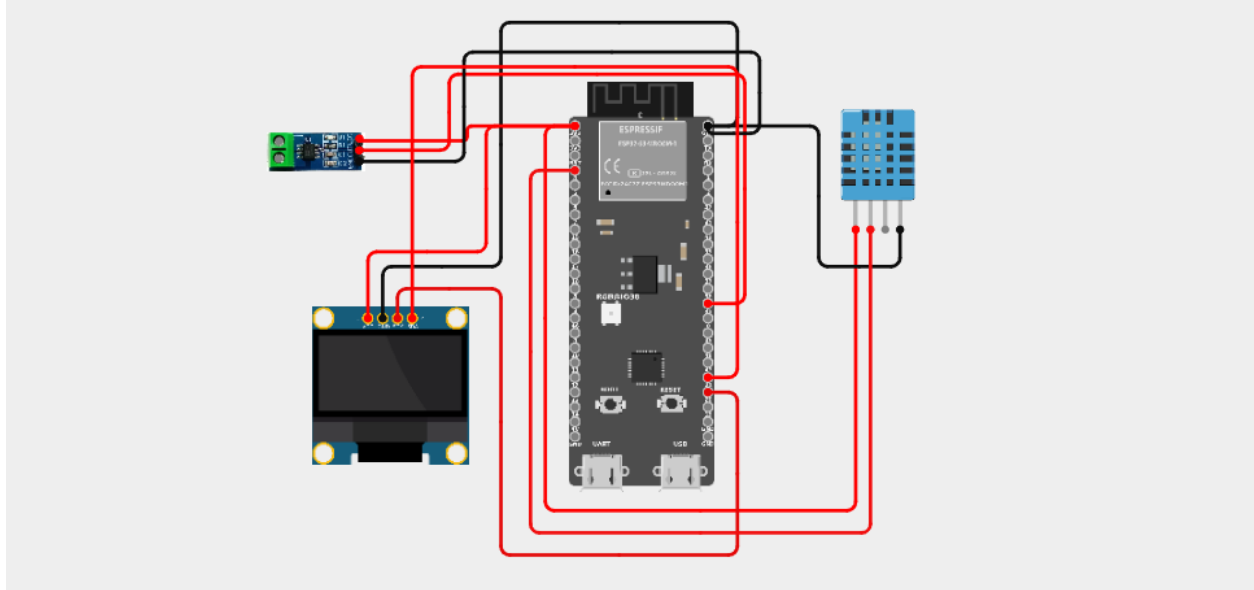
ELEVATOR OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Voltage	Normal rated range	Slight deviation	High/low abnormal
Current	Normal load	High load	Overcurrent
Temperature	Normal	Increased	High temperature
Humidity	Normal	Increased	Very high
DG Status	Normal	Warning	Critical

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Voltage	Voltage Sensor Module	Measure DG output voltage
Current	ACS712	Measure load current
Temperature	DHT11	Monitor temperature
Humidity	DHT11	Monitor environmental humidity
DG Status	ESP32	Determine operating condition
Display	OLED	Show real-time values
Cloud Data	Wi-Fi/Cloud	Remote monitoring

CIRCUIT DESIGN



ALGORITHM

- START
- Initialize ESP32.
- Initialize ACS712 current sensor.
- Initialize voltage sensor.
- Initialize DHT11.
- Initialize OLED display.
- Connect ESP32 to Wi-Fi.
- Read DG voltage.
- Read DG current.
- Read temperature and humidity.
- Process the sensor values.
- Display values on OLED.
- Send data to cloud.
- Compare measured values with the defined limits.
- If values are normal, display NORMAL.
- If values exceed warning limits, display WARNING.
- If critical limits are exceeded, display CRITICAL.
- Continue monitoring.
- STOP/REPEAT

CODING

```
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

// =====
// WIFI DETAILS
// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO DETAILS
// =====

#define IO_USERNAME  "Hair_13"
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// Create Adafruit IO WiFi object
AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASSWORD
);

// =====
// ADAFRUIT IO FEEDS
// =====

AdafruitIO_Feed *currentFeed =
  io.feed("dg-current");

AdafruitIO_Feed *temperatureFeed =
  io.feed("dg-temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("dg-humidity");

AdafruitIO_Feed *runtimeFeed =
  io.feed("dg-runtime");
```

```

AdafruitIO_Feed *statusFeed =
  io.feed("dg-status");

AdafruitIO_Feed *alertFeed =
  io.feed("dg-alert");

// =====
// ACS712 CURRENT SENSOR
// =====

#define CURRENT_PIN 34

// ACS712 30A sensitivity = 66 mV/A
// If using ACS712 20A, use 0.100
// If using ACS712 5A, use 0.185

#define ACS712_SENSITIVITY 0.066

// ESP32 ADC parameters
#define ADC_REFERENCE 3.3
#define ADC_RESOLUTION 4095

// Zero-current voltage
// Adjust during calibration if required
float zeroCurrentVoltage = 1.65;

// =====
// DHT11
// =====

#define DHT_PIN 4
#define DHT_TYPE DHT11

DHT dht(
  DHT_PIN,
  DHT_TYPE
);

// =====
// OLED
// =====

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(

```

```

    SCREEN_WIDTH,
    SCREEN_HEIGHT,
    &Wire,
    -1
);

// =====
// DG VARIABLES
// =====

float current = 0.0;

float temperature = 0.0;

float humidity = 0.0;

float runtimeHours = 0.0;

String dgStatus = "NORMAL";

String dgAlert = "NONE";

// =====
// RUNTIME
// =====

unsigned long dgStartTime = 0;

// =====
// CLOUD TIMING
// =====

unsigned long lastSend = 0;

const unsigned long sendInterval = 15000;

// =====
// CURRENT SENSOR SETTINGS
// =====

const int currentSamples = 200;

// =====
// CURRENT READING FUNCTION
// =====

```

```

float readCurrent()
{
    float sum = 0.0;

    for (int i = 0; i < currentSamples; i++)
    {
        int rawValue = analogRead(CURRENT_PIN);

        float voltage =
            (rawValue * ADC_REFERENCE) /
            ADC_RESOLUTION;

        sum += voltage;

        delayMicroseconds(100);
    }

    float averageVoltage =
        sum / currentSamples;

    float currentValue =
        (averageVoltage - zeroCurrentVoltage) /
        ACS712_SENSITIVITY;

    if (currentValue < 0)
    {
        currentValue =
            -currentValue;
    }

    // Remove small sensor noise
    if (currentValue < 0.20)
    {
        currentValue = 0.0;
    }

    return currentValue;
}

// =====
// READ DHT11
// =====

void readTemperatureHumidity()
{
    float newTemperature =

```

```

    dht.readTemperature();

    float newHumidity =
    dht.readHumidity();

    if (!isnan(newTemperature))
    {
        temperature =
        newTemperature;
    }

    if (!isnan(newHumidity))
    {
        humidity =
        newHumidity;
    }
}

// =====
// CALCULATE DG RUNTIME
// =====

void calculateRuntime()
{
    unsigned long elapsedTime =
    millis() - dgStartTime;

    runtimeHours =
    elapsedTime / 3600000.0;
}

// =====
// DG STATUS AND ALERT
// =====

void checkDGStatus()
{
    dgStatus = "NORMAL";

    dgAlert = "NONE";

    // High current condition
    if (current > 30.0)
    {
        dgStatus = "CRITICAL";
    }
}

```

```

    dgAlert = "HIGH CURRENT";
}

// High temperature condition
else if (temperature > 80.0)
{
    dgStatus = "CRITICAL";

    dgAlert = "HIGH TEMPERATURE";
}

// Warning temperature
else if (temperature > 65.0)
{
    dgStatus = "WARNING";

    dgAlert = "TEMPERATURE HIGH";
}

// DG is running
else if (current > 1.0)
{
    dgStatus = "RUNNING";

    dgAlert = "NONE";
}

// DG is not drawing current
else
{
    dgStatus = "IDLE";

    dgAlert = "NONE";
}
}

// =====
// OLED DISPLAY
// =====

void updateOLED()
{
    display.clearDisplay();

    display.setTextColor(
        SSD1306_WHITE

```

```
);  
  
display.setTextSize(1);  
  
// Title  
display.setCursor(0, 0);  
  
display.println(  
    "INDUSTRIAL DG MONITOR"  
);  
  
// Current  
display.setCursor(0, 13);  
  
display.print("Current: ");  
  
display.print(  
    current,  
    1  
);  
  
display.println(" A");  
  
// Temperature  
display.setCursor(0, 24);  
  
display.print("Temp: ");  
  
display.print(  
    temperature,  
    1  
);  
  
display.println(" C");  
  
// Humidity  
display.setCursor(0, 35);  
  
display.print("Humidity: ");  
  
display.print(  
    humidity,  
    0  
);  
  
display.println(" %");
```

```

// Runtime
display.setCursor(0, 46);

display.print("Runtime: ");

display.print(
  runtimeHours,
  2
);

display.println(" H");

// Status
display.setCursor(0, 57);

display.print(
  dgStatus
);

display.display();
}

// =====
// SEND DATA TO ADAFRUIT IO
// =====

void sendToAdafruitIO()
{
  if (io.status() < AIO_CONNECTED)
  {
    Serial.println(
      "Adafruit IO not connected."
    );

    return;
  }

  Serial.println();

  Serial.println(
    "Sending DG data to Adafruit IO..."
  );

  // Current
  currentFeed->save(

```

```

        current
    );

    // Temperature
    temperatureFeed->save(
        temperature
    );

    // Humidity
    humidityFeed->save(
        humidity
    );

    // Runtime
    runtimeFeed->save(
        runtimeHours
    );

    // Status
    statusFeed->save(
        dgStatus
    );

    // Alert
    alertFeed->save(
        dgAlert
    );

    Serial.println(
        "DG data sent successfully."
    );
}

// =====
// SERIAL MONITOR
// =====

void printDGData()
{
    Serial.println();

    Serial.println(
        "=====
    );

    Serial.println(

```

```
"    INDUSTRIAL DG MONITOR"
);

Serial.println(
    "=====
");

Serial.print(
    "Current    : "
);

Serial.print(
    current,
    2
);

Serial.println(
    " A"
);

Serial.print(
    "Temperature : "
);

Serial.print(
    temperature,
    2
);

Serial.println(
    " C"
);

Serial.print(
    "Humidity    : "
);

Serial.print(
    humidity,
    2
);

Serial.println(
    " %"
);
```

```

Serial.print(
  "Runtime      : "
);

Serial.print(
  runtimeHours,
  2
);

Serial.println(
  " hours"
);

Serial.print(
  "DG Status    : "
);

Serial.println(
  dgStatus
);

Serial.print(
  "DG Alert     : "
);

Serial.println(
  dgAlert
);

Serial.println(
  "=====
);
}

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(
    115200
  );

  delay(1000);

```

```
Serial.println();

Serial.println(
    "=====
");

Serial.println(
    "SYSTEM READY"
);

Serial.println(
    "=====
");

Serial.println(
    "Industrial DG monitoring is active."
);

// =====
// ADC
// =====

analogReadResolution(
    12
);

// =====
// CURRENT SENSOR
// =====

pinMode(
    CURRENT_PIN,
    INPUT
);

// =====
// DHT11
// =====

dht.begin();

// =====
// OLED
// =====

Wire.begin(
```

```
    21,  
    22  
);  
  
if (!display.begin(  
    SSD1306_SWITCHCAPVCC,  
    0x3C  
))  
{  
    Serial.println(  
        "OLED not detected!"  
    );  
}  
else  
{  
    display.clearDisplay();  
  
    display.setTextColor(  
        SSD1306_WHITE  
    );  
  
    display.setTextSize(1);  
  
    display.setCursor(  
        10,  
        15  
    );  
  
    display.println(  
        "DG MONITORING"  
    );  
  
    display.setCursor(  
        10,  
        30  
    );  
  
    display.println(  
        "SYSTEM READY"  
    );  
  
    display.display();  
  
    delay(2000);  
}
```

```

// =====
// START DG RUNTIME
// =====

dgStartTime =
  millis();

// =====
// ADAFRUIT IO CONNECTION
// =====

Serial.println();

Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();

// =====
// NON-BLOCKING CONNECTION WAIT
// =====

unsigned long connectionStart =
  millis();

while (
  io.status() < AIO_CONNECTED &&
  millis() - connectionStart < 20000
)
{
  Serial.print(".");

  delay(500);
}

Serial.println();

if (io.status() >= AIO_CONNECTED)
{
  Serial.println(
    "Connected to Adafruit IO!"
  );

  display.clearDisplay();
}

```

```
display.setTextSize(1);

display.setCursor(
  0,
  15
);

display.println(
  "Adafruit IO"
);

display.setCursor(
  0,
  30
);

display.println(
  "CONNECTED"
);

display.display();

delay(1500);
}
else
{
  Serial.println(
    "Adafruit IO connection failed."
  );

  Serial.println(
    "System will continue locally."
  );
}

Serial.println();

Serial.println(
  "Current monitoring is active."
);
}

// =====
// LOOP
// =====
```

```

void loop()
{
  // Keep Adafruit IO alive
  io.run();

  // =====
  // READ CURRENT
  // =====

  current =
    readCurrent();

  // =====
  // READ TEMPERATURE AND HUMIDITY
  // =====

  readTemperatureHumidity();

  // =====
  // CALCULATE RUNTIME
  // =====

  calculateRuntime();

  // =====
  // CHECK STATUS
  // =====

  checkDGStatus();

  // =====
  // UPDATE OLED
  // =====

  updateOLED();

  // =====
  // SERIAL OUTPUT
  // =====

  printDGData();

  // =====
  // SEND TO CLOUD
  // =====

```

```

if (
  millis() - lastSend >=
  sendInterval
)
{
  lastSend =
  millis();

  sendToAdafruitIO();
}

// Short delay
delay(1000);
}

```

SYSTEM WORKING

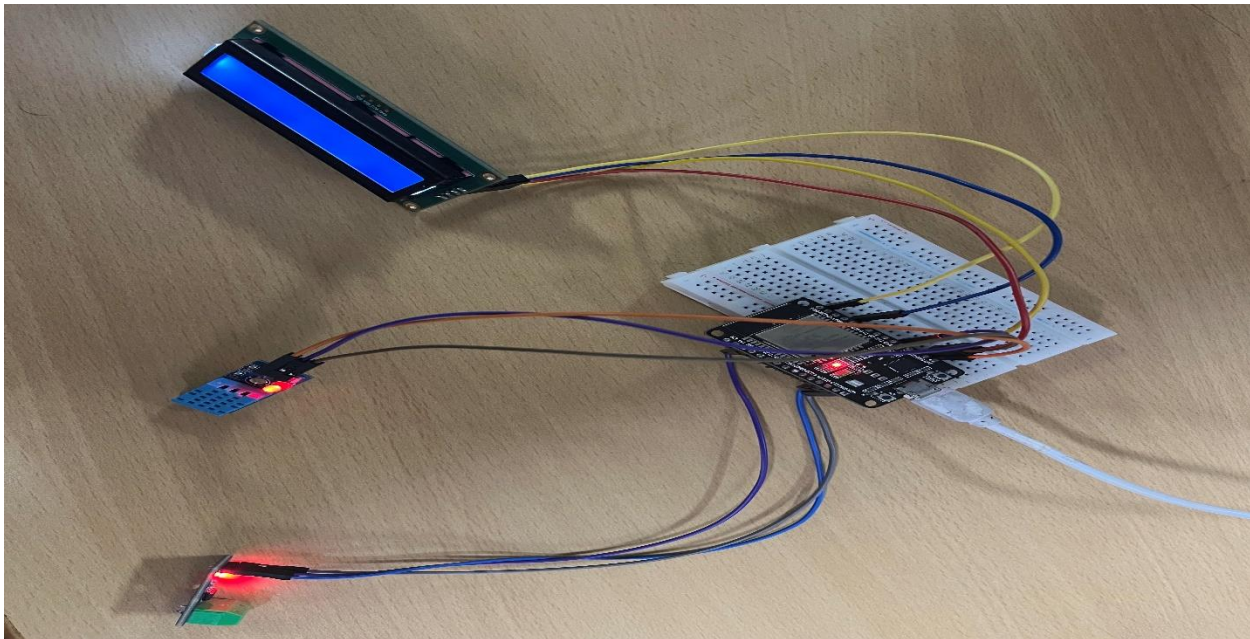
The ESP32 continuously collects DG performance data from the connected sensors. The **ACS712** measures current, while the voltage sensor measures DG voltage. The DHT11 monitors temperature and humidity. The ESP32 processes these values and displays them on the OLED. Through the built-in Wi-Fi, the measured data is transmitted to the cloud platform. The user can remotely observe the DG operating parameters through the cloud dashboard. If abnormal conditions such as excessive current, abnormal voltage, or high temperature are detected, the system can indicate a warning or critical condition.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	ESP32 WROOM-32	1	Main controller
2	ACS712 Current Sensor	1	Current measurement
3	Voltage Sensor Module	1	Voltage measurement
4	DHT11 Sensor	1	Temperature & humidity
5	OLED Display	1	Local parameter display
6	Wi-Fi	Built-in ESP32	Cloud communication
7	Cloud Platform	1	Remote monitoring
8	Power Supply	1	Power source
9	Breadboard & Jumper Wires	1 set	Circuit connection

PROTOTYPE IMPLEMENTATION

The prototype is designed using an **ESP32 WROOM-32 microcontroller**, **ACS712 current sensor**, **voltage sensor module**, **DHT11 temperature and humidity sensor**, and **OLED display**. The voltage sensor and ACS712 measure the DG electrical parameters, while the DHT11 monitors temperature and humidity. The ESP32 processes the sensor data and displays the values on the OLED. Using the built-in Wi-Fi, the collected DG performance data is transmitted to a cloud platform for remote monitoring and analytics. The prototype can be implemented on a breadboard with suitable power supply and jumper-wire connections.

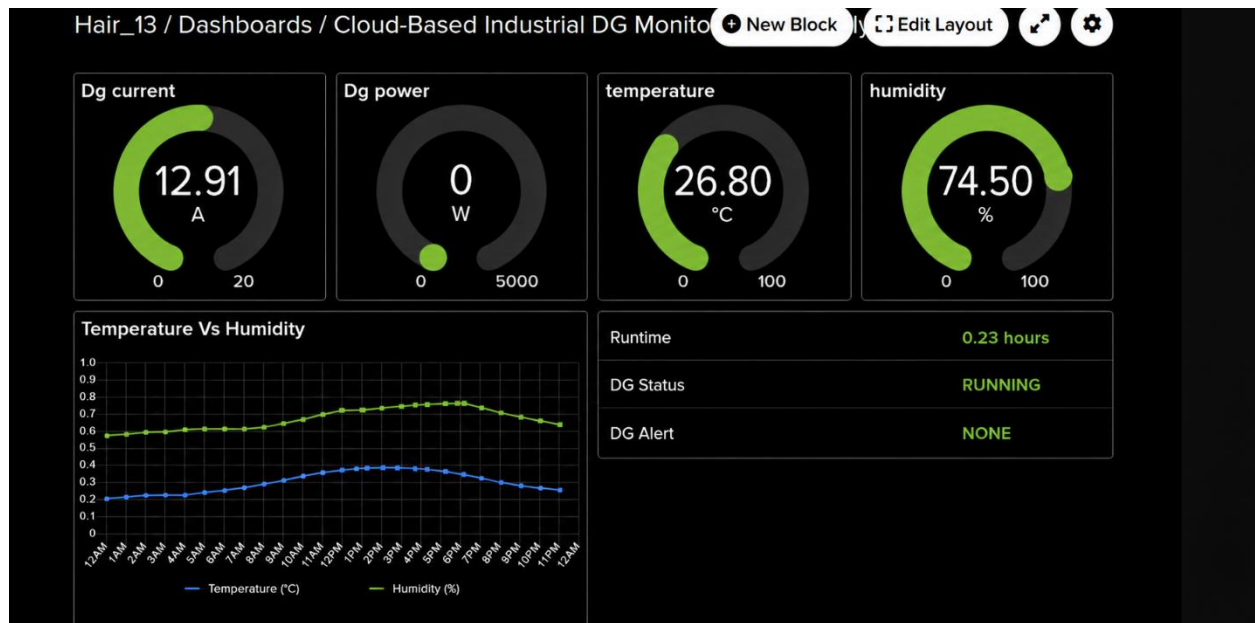


RESULTS & PERFORMANCE ANALYSIS

Condition	Sensor Output	System Action
DG OFF	No/low output	DG OFF
DG operating normally	Normal voltage/current	NORMAL
High load	Increased current	WARNING
Overcurrent	Very high current	CRITICAL
Normal temperature	Normal temperature	NORMAL
High temperature	Increased temperature	WARNING/CRITICAL
Wi-Fi connected	Data available	Cloud monitoring
Wi-Fi disconnected	Communication unavailable	Local OLED monitoring

PERFORMANCE

- Real-time DG parameter monitoring.
- Remote cloud-based monitoring.
- Voltage measurement.
- Current measurement.
- Temperature monitoring.
- Humidity monitoring.
- OLED-based local display.
- Abnormal-condition identification.
- Continuous data collection.
- Improved DG monitoring and maintenance.
- Supports industrial remote monitoring.



iotexp-2.ino

```
729
730   printDGData();
731
732   // =====
733   // SEND TO CLOUD
734   // =====
735
736   if (
737     millis() - lastSend >=
738     sendInterval
739   )
740   {
741     lastSend =
742       millis();
743     sendToAdafruitIO();
744   }
745
```

Output Serial Monitor X

Message (Enter to send message to 'ESP32 Dev Module' on 'COM8')

New Line 1152

```
Current      : 12.91 A
Temperature  : 27.80 C
Humidity     : 72.30 %
Runtime      : 0.06 hours
DG Status    : RUNNING
DG Alert     : NONE
```